VetCare

Veterinary Clinic Management System

A database-focused web application built with Python, Flask, and PostgreSQL. The project demonstrates relational database design, SQL programming, trigger-based validation, and a browser interface for working with veterinary and product-order data.

Project type	Academic project / technical portfolio
Backend	Python, Flask, SQLAlchemy SQL execution
Database	PostgreSQL
Interface	HTML forms with JavaScript fetch requests
Main focus	Relational modelling, SQL functions, views, constraints, and triggers

Portfolio summary

VetCare combines a structured PostgreSQL database with Flask endpoints and a browser-based user interface. It is designed to manage users, pets, clinics, veterinarians, appointments, health records, vaccinations, products, orders, and order items. It also demonstrates database-side business rules through SQL functions and triggers.

Prepared as a technical project report. Database structure and implemented behaviour are described from the project information available at the time of writing. Screenshots and full source listings can be added in a later revision.

1. Project Overview

VetCare is a small web application for a veterinary clinic environment. Its database stores information about pet owners, pets, clinics, veterinarians, appointments, vaccinations, health records, products, and customer orders. The application provides browser forms and Flask routes that communicate with PostgreSQL.

Project goals

- Design a relational database with primary keys, foreign keys, unique constraints, and check constraints.
- Connect a Python Flask backend to PostgreSQL and execute parameterised SQL statements.
- Display query results and function outputs in a browser interface.
- Demonstrate views, aggregate queries, set operations, SQL functions, and database triggers.
- Validate input and return clear success or error messages to the user.

2. Technology Stack

Technology	Role in the project
Python	Backend language and input validation.
Flask	HTTP routes and server-side request handling.
PostgreSQL	Relational data storage, SQL functions, views, constraints, and triggers.
SQLAlchemy	Database engine and execution of SQL statements from Flask.
HTML and CSS	Browser forms and page layout.
JavaScript Fetch API	Sends form data to Flask endpoints and displays JSON responses.
pgAdmin / Postman	Database inspection and API testing during development.

3. Database Design

The schema contains ten main tables. Primary keys identify rows, while foreign keys connect related records. The design separates core entities (users, pets, clinics, veterinarians, products) from activity and transaction tables (appointments, vaccinations, health records, orders, and order items).

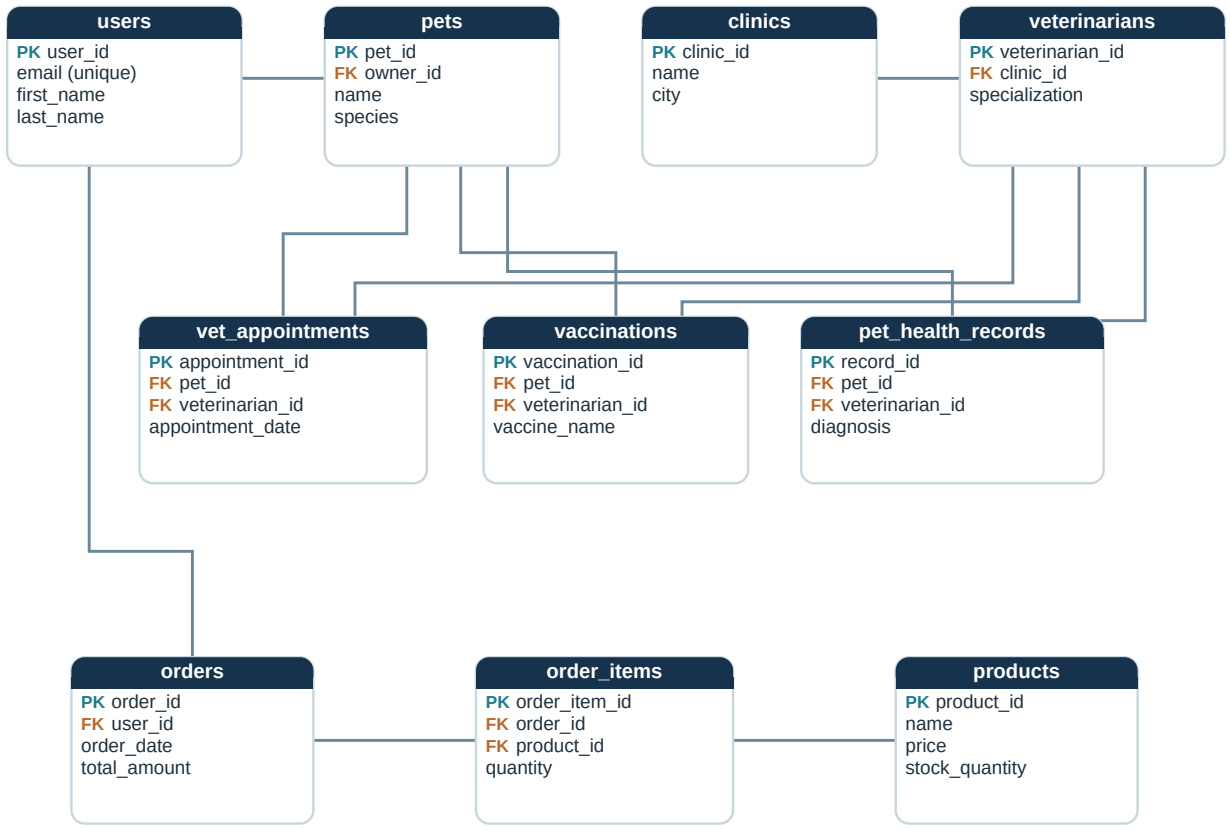
Table	Purpose	Key relationships
users	Stores customer/owner details.	Referenced by pets and orders.
pets	Stores pet details and owner link.	References users; referenced by appointments, vaccinations, health records.
clinics	Stores clinic contact and opening-hour details.	Referenced by veterinarians.
veterinarians	Stores veterinarian details and specialisation.	References clinics; referenced by appointments, vaccinations, health records.
vet_appointments	Stores appointments, date/time, reason, and status.	References pets and veterinarians.
vaccinations	Stores vaccine and due-date records.	References pets and veterinarians.
pet_health_records	Stores diagnoses, symptoms, and treatments.	References pets and veterinarians.
products	Stores product details, price, and stock quantity.	Referenced by order_items.
orders	Stores customer orders and total amounts.	References users; referenced by order_items.
order_items	Stores products and quantities within an order.	References orders and products.

Important constraints

- **Primary keys:** for example, `users.user_id`, `pets.pet_id`, and `products.product_id`.
- **Foreign keys:** link dependent records to their parent rows and help preserve referential integrity.
- **Unique constraints:** prevent duplicate user emails and duplicate pet microchip numbers.
- **Check constraints:** product prices and stock quantities cannot be negative; appointment status is restricted to defined values.
- **Delete rules:** selected foreign keys use `ON DELETE CASCADE` so dependent rows are removed with their parent.

4. Entity Relationship Diagram

The diagram below summarises the main foreign-key relationships. It is a compact portfolio view of the schema; the SQL scripts remain the authoritative source for complete column definitions and constraints.



PK = Primary Key | FK = Foreign Key | Lines show foreign-key relationships

Relationship overview: one user can own multiple pets and place multiple orders; one clinic can employ multiple veterinarians; pets and veterinarians are linked to appointment, vaccination, and health-record entries; orders and products are connected through order_items.

5. SQL Queries and Database Features

The project includes SQL work beyond basic data insertion and selection. These features help demonstrate practical database skills.

Views

A view is used to present completed veterinarian appointments through a report endpoint. Views can package a reusable query and make reporting code easier to maintain.

Aggregate functions and HAVING

A best-selling products report groups order items and filters grouped results with HAVING. This demonstrates aggregation over transactional data.

UNION

A report endpoint uses UNION to combine results from two product queries into one result set.

Parameterized SQL

Flask routes pass values as SQL parameters instead of building query strings by concatenating user input.

Sequences / SERIAL

ID columns use PostgreSQL SERIAL behaviour so the database can generate identifiers automatically when new records are inserted.

Transactions

The order-item insertion route uses a database transaction. If insertion or a trigger fails, the transaction can roll back instead of leaving partial changes.

6. SQL Functions

Function	Purpose	UI/API usage
calculate_customer_spending(customer_id)	Calculates the total spending associated with a customer.	POST /functions/customer-spending
calculate_order_total(id_order)	Calculates an order total from its order items.	POST /functions/order-total
check_product_stock(pro_id, requested_quantity)	Checks whether a requested quantity is available.	POST /functions/check-stock
get_customer_orders(customer_id)	Returns a customer's orders; the implementation uses a cursor and a RECORD variable.	POST /functions/customer-orders

The project has three core parameterised business functions plus an additional customer-orders function for the cursor/RECORD requirement.

7. Triggers and Business Rules

Database triggers enforce rules close to the data. This means the rules still apply when an insert is made through the web interface or directly through SQL.

Trigger / function	Event and behaviour	Test through the UI
trig_negative_stock / prevent_negative_stock()	BEFORE INSERT on order_items. Rejects quantities that are not positive and requests that exceed current stock.	Submit an order item with quantity 0 or greater than available stock; insertion should fail.
trig_product_stock / update_product_store()	AFTER INSERT on order_items. Decreases the related product stock by the ordered quantity.	Submit a valid order item and check the stock quantity in PostgreSQL.
trig_order_total / update_order_total()	AFTER INSERT on order_items. Recalculates the order total using its item quantities and unit prices.	Submit a valid order item and check the order total.
trig_prevent_invalid_appdate / invalid_appointment_dates()	BEFORE INSERT or UPDATE on vet_appointments. Rejects appointment dates earlier than the current date.	Submit a past date and confirm that the UI displays the database error.

Testing note: The order-item and appointment forms have been tested in the browser according to the project development notes. The order-item form accepts a unit price from the user at present; using the product's stored price automatically is a possible improvement.

8. Web Application Structure

Component	Responsibility
app.py	Defines Flask routes for product operations, reports, SQL functions, order-item creation, and appointment creation.
db.py	Initialises the database connection using configuration from an environment file.
templates/index.html	Contains browser forms for product operations, reports, functions, order items, and appointments.
PostgreSQL database	Stores the records and enforces constraints, functions, views, and triggers.

Main application flows

- A user submits an HTML form in the browser.
- JavaScript sends the form values to a Flask endpoint as JSON.
- Flask validates the input and executes parameterised SQL through the database engine.
- PostgreSQL returns the result or raises a constraint/trigger error.
- The endpoint responds with JSON, and the browser displays the message.

9. Development and Test Evidence

During development, endpoints were exercised using the browser and Postman, while table structures and row counts were checked in pgAdmin. The latest recorded counts for the ten main tables are shown below. The clinics table currently contains six records; the other nine main tables contain at least ten records.

Table	Rows	Table	Rows
clinics	6	orders	13
users	10	order_items	23
veterinarians	10	products	13
pets	12	vet_appointments	14
vaccinations	12	pet_health_records	13

Suggested evidence screenshots for a final submission

- The home page showing the application sections.
- A successful product insert/update/delete operation.
- The completed appointments view and the best-selling products report.
- A successful SQL function result, including customer orders.
- A valid order item that updates stock and order total.
- An invalid order-item quantity or insufficient-stock response.
- A rejected appointment with a past date.
- pgAdmin screenshots of table relationships, constraints, functions, and triggers.

10. Setup Notes

The source project should include the Flask code, HTML templates, SQL scripts, and a README. A local setup normally requires Python, PostgreSQL, the required Python packages, and a database created from the supplied SQL scripts.

- Create the PostgreSQL database and run the SQL scripts in the correct order.
- Create a local environment file containing the database connection string. Do not publish this file.
- Install the project's Python dependencies in a virtual environment.
- Start the Flask application and open the local address in a browser.
- Use the README to record the exact commands and dependency versions used by the final project.

> **Security note: Never commit database passwords, secret keys, or personal credentials to a public repository. Include an example environment file with placeholder values instead of the real .env file.**

11. Possible Future Improvements

- Read the product price from the products table instead of allowing the order-item form to submit any unit price.
- Improve error handling by returning clear messages for invalid IDs, missing records, and database constraint failures.

- Add authentication and role-based access control if the application is extended beyond a course project.
- Add automated tests for API routes, SQL functions, and trigger behaviour.
- Improve UI validation and add clearer status indicators for successful and failed operations.
- Add screenshots and final database export evidence before publishing the repository.

Portfolio description: VetCare is a Python Flask and PostgreSQL web application that demonstrates relational database design, SQL reporting, parameterised functions, database triggers, and a browser-based interface for veterinary and product-order workflows.